Guide d'intégration BO6 — M7 « Catalogue local » et M14 « Stylist / boutique mode »

Pour la personne qui intègre les modules DressMe. Rédigé le 21/10/2026 par BO6 (kenzabokri). Branche : `BO6/local_catalogue` .

---

0. En bref

	M7 Catalogue local	M14 Stylist / boutique mode
Rôle	Chercher des pièces dans 43 boutiques tunisiennes (6 375 produits, prix en TND)	Côté professionnel : boutiques et stylistes, tableau de bord de la demande, hand-off assistant → styliste, conversation privée, looks
Agent	**`localshop`** = l'agent BO6 (un agent par BO). `tool.py` trouvé par `orchestrator/registry.py`	**Pas d'agent** : compétence `boutique` de `localshop` . Pas de `tool.py`
État	Réel et complet (données, FashionCLIP, évaluation, mémoire, `next_actions`)	Réel pour M7 ; **M1, M2, M3, M5, M6, M10 sur mock** tant qu'ils ne sont pas fusionnés
LLM	**Qwen 2.5 7B en local (Ollama)**, un seul pour M7 et M14 (`m07_local_catalogue/llm.py`) : comprend les requêtes difficiles	Le même LLM : résumé pour la styliste, escalade, conseils boutique
Tests	`pytest modules/m07_local_catalogue` (156)	`pytest modules/m14_stylist_mode` (40)
App	`/#/bo6_local_market`	`/#/bo6_boutique` (Espace pro) et `/#/bo6_boutique/help/:pro/:client`

Ce qu'il reste à faire pour M14, par priorité (détails au §4) : 1. Un **identifiant utilisateur commun** à tous les modules, puis l'**authentification** (JWT). 2. Passer **M1, M3, M2, M6, M10** en réel : un seul changement dans `config.yaml` . 3. **M5** : se mettre d'accord avec BO4 sur le format du profil (M5 n'existe pas encore). 4. **Orchestrateur (BO8)** : le bouton « Parler à un styliste » dans le chat → `POST /m14/handoff` . 5. Remplacer le stockage JSON par **Postgres**. 6. **App côté client** : consentement, looks reçus, conversation. 7. **LLM** : installer Ollama et `qwen2.5:7b` sur la machine qui fait tourner l'API (§4.11). Sans lui, tout marche quand même (règles).

---

1. Où est le code

```
backend/modules/m07_local_catalogue/   M7 (agent localshop)
backend/modules/m14_stylist_mode/      M14
mobile/lib/features/bo6_local_market/  écran « Boutiques locales » (M7)
mobile/lib/features/bo6_boutique/      « Espace pro » + page « Aider un client » (M14)
mobile/tool/serve_app.py               sert l'app web + l'API sur un seul port (pas de CORS)
ml/data/shops/                         catalogue, photos (327 Mo), index FashionCLIP : versionnés dans git
ml/data/bo6/, ml/data/logs/            mémoire, données M14, journaux : JAMAIS dans git (recréés à l'exécution)
rapport/                               rapports M7 et M14 (md + pdf)
```

Fichiers partagés modifiés par BO6 (attention aux conflits, §5) : `mobile/lib/app/router.dart` (3 routes ajoutées) et `mobile/tool/serve_app.py`. Rien dans `backend/shared/` ni dans `backend/orchestrator/`.

---

2. M7 — Catalogue local (agent `localshop`)

2.1 Ce qu'il fait

`"baskets blanches à Tunis max 80 TND"` (français, anglais, darija, arabe) → compréhension de la requête → recherche hybride (FashionCLIP texte→photo + BM25) → filtres (prix, ville, type, couleur, genre, stock) → score + diversité → résultats avec boutique, prix en TND, photo, lien et raisons.

2.2 Endpoints (`/m07`)

Endpoint	Rôle
`POST /m07/search`	Corps `LocalShopInput`, réponse `LocalShopOutput` (+ `search_id`)
`GET /m07/card` · `GET /m07/identity`	Fiche de l'agent (lue par l'orchestrateur)
`POST /m07/events`	Clics de l'app : `product_view`, `shop_click` (alimente le tableau de bord M14)
`POST /m07/feedback` · `GET /m07/history/{user}` · `DELETE /m07/memory/{user}`	Mémoire long terme, droit à l'oubli
`GET /m07/shops` · `GET /m07/products/{id}` · `GET /m07/images/{name}`	Boutiques, un produit, photo locale

2.3 Contrat (champs utiles à l'intégration)

- **Entrée** : `query` et/ou `gap` / `category` / `subcategory` / `image_embedding` (512, FashionCLIP) ; filtres `city`, `max_price_tnd`, `min_price_tnd`, `color`, `gender` ; `user_id`, `style_vector` (id M5 ou 512 valeurs) ; `k`, `offset`.
- **Champs agent (tous optionnels)** : `session_id`, `language` (`fr` / `en` / `ar`), `answer` (`{question_key, value}` = réponse à la question précédente), `next_page`.
- **Sortie** : `results[]`, `parsed`, `total_matches`, `note`, `search_id`, et pour l'orchestrateur : `agent`, `decision_id`, `question` + `question_key`, `memory_notes`, `next_actions[]`.

- `next_actions` : `ask_user` , `open_shop` , `judge` → **smartbuy**, `show_outfits` → **stylist**, `more_results` → localshop, `feedback` .
- **Exemple dans l'app** (écran « Local shops », client de démo `usr_042`) : « jean » → carte « The agent asks: For women or for men? » ; un clic envoie `answer` dans la même `session_id` (130 → 79 jeans femme) et la réponse est gardée en mémoire (« Forget my answers » = `DELETE /m07/memory/{user}`). Dans la fiche d'un produit : « Talk to a stylist » crée un vrai hand-off M14 ; « Is it a good buy? » montre ce qui est passé à Smart Buy (`judge`), dont l'écran BO5 n'est pas encore fusionné. C'est ce que l'assistant BO8 doit faire avec `question` et `next_actions` .

2.4 Qui appelle M7 aujourd'hui

- **Orchestrateur** : `tool.py` (nom `localshop` , description construite depuis `identity.py`).
- **Smart Buy (BO5)** : en process `m7.service.search(body, channel="smartbuy")` ou en HTTP `POST /m07/search` . Les champs ajoutés sont optionnels, donc rien n'est cassé.
- **BO7** (`feat/bo7-integration`) : pièces des boutiques locales pour les looks.

2.5 À savoir pour faire tourner M7

- **Données** : `ml/data/shops/` est dans git (CSV nettoyé, index, photos). Rien à télécharger.
- **Modèle** : FashionCLIP (`patrickjohncyh/fashion-clip` , environ 600 Mo) se télécharge depuis Hugging Face à la première recherche (environ 15 s de chargement ensuite). On peut aussi le placer dans `ml/models/fashion-clip/` .
- **Dépendances** : `backend/modules/m07_local_catalogue/requirements.txt` (torch, transformers).
- **Sans données** (clone sans le catalogue), le service répond avec `mock.py` , sans planter.

---

3. M14 — Stylist / boutique mode

3.1 Fonctions et endpoints (`/m14`)

Fonction	Endpoints	Pour
F1 Comptes pro	`POST/GET /m14/pro` , `GET/PATCH /m14/pro/{id}`	tous
F2 Consentement + fiche	`POST /m14/clients` , `POST /m14/clients/{link}/consent` , `DELETE /m14/clients/{link}` , `GET /m14/clients` , `GET /m14/clients/{client}/brief` , `GET /m14/clients/{client}/wardrobe`	BO4 BO5 BO7
F2b Conversation privée	`GET/POST/DELETE /m14/chat/{pro}/{client}`	BO6 BO8
F3 Relecture du profil	`POST /m14/clients/{client}/review` , `GET /m14/reviews/agreement`	BO4
F4 Looks	`POST/GET /m14/looks`	BO7
F5 Vendeur en magasin	`POST /m14/instore/session`	BO5
F6 Tableau de bord	`GET /m14/dashboard?shop=&days=&source=` , `GET /m14/pro/{id}/dashboard`	BO6

Fonction	Endpoints	Pour
F7 Hand-off	`POST /m14/handoff/check`, `POST /m14/handoff`, `GET /m14/handoff/queue`, `GET/PATCH /m14/handoff/{id}`	BO8

Formes exactes : `contract.py`. Réponses fixes pour les tests des autres BO : `mock.py`.

3.2 Données de M14

- `ml/data/bo6/m14/*.json` : `pros`, `links`, `handoffs`, `looks`, `reviews`, `chats`. Interface `store.Collection` (`all / get / put / delete / find`).
- `ml/data/logs/m14_simulated_events.jsonl` : trafic simulé, séparé du vrai journal M7 (`ml/data/logs/m07_events.jsonl`). Le tableau de bord lit les deux et affiche la part simulée.
- Données de démo (non versionnées, à recréer) : `bash cd backend python -m modules.m14_stylist_mode.simulate.pros --reset # 8 stylistes, 4 boutiques, clients usr_042..044 python -m modules.m14_stylist_mode.simulate.traffic # 12 semaines de recherches via la vraie M7 (~2 min)`

3.3 Comment M14 appelle les autres modules

Tout passe par `clients.py`. Pour chaque module : appel réel s'il est listé dans `config.yaml → real`, sinon (ou en cas d'erreur) réponse mock. Chaque réponse de M14 indique `sources: {m1: real|mock, …}`.

Module	Appel fait par M14	Ce que M14 lit	Aujourd'hui
M7 (BO6)	en process : `m7.service.product/shops/memory()`	produits, boutiques, recherches du client	**réel**
M1 (BO3, `garde-robe`)	`POST /m01/items {user_id}`	`items[]` : `item_id`, `category`, `subcategory`, `colors[0].name`, `wear_count`, `price_tnd`, `image_cutout` / `image_raw`	mock
M3 (BO1/2)	`POST /m03/stats {user_id, payload: {item_ids}}`	`result.items[].wear_count`	mock
M5 (BO4)	`POST /m05/profile`, `/m05/gaps`, `/m05/feedback`	voir §4.4	mock (M5 n'existe pas)
M2 (BO1/2)	`POST /m02/score {user_id, payload: {items, outfits:[[ids]]}}`	`result.outfits[0].score`	mock
M6 (BO5)	`POST /m06/verdict {user_id, probe_image, price_tnd, shop_brand, language, session_id}`	`verdict`, `score`, `reasons`, `explanation`	mock
M10 (BO7)	`POST /m10/cards {user_id, kind:"outfit", caption, language, items:[CardItem]}`	`image_url`, `share_url`	mock

Les corps envoyés à **M2** et **M10** ont été validés contre leurs vrais contrats (branches `feat/m02-m03-m04-m17` et `feat/bo7-m10-m15`). Pour M2, les pièces sont envoyées **en ligne** (`payload.items`, format `Garment` partagé) : M2 ne note que les pièces qu'on lui donne, et les pièces de boutique ne sont pas dans la garde-robe du client.

4. Ce qu'il faut faire pour M14

4.1 Utilisateurs : un identifiant commun (à faire en premier)

- **Le `client_id` de M14 doit être le `user_id` de M1, M3, M5 et M7.** C'est ce qui relie la fiche, la garde-robe, l'historique de recherche et la conversation. Aujourd'hui, les clients de démo (`usr_042` , `usr_043` , `usr_044`) n'existent pas dans M1.
- Pour tester : créer dans M1 la garde-robe d'un client test avec le **même** `user_id` , ou utiliser un utilisateur de démo de M1 comme client dans M14.
- **Comptes pro** : un `pro_id` M14 doit être rattaché à un utilisateur (styliste ou propriétaire de boutique). À prévoir : `user_id` dans `Pro` , et le rôle `stylist` / `boutique` dans les comptes.
- **Authentification (JWT, `shared/config.py → jwt_secret`)**. Aujourd'hui, l'identité est **déclarée par l'appelant** dans ces paramètres ; ils devront venir du jeton :

Endpoint	Paramètre à remplacer par l'identité du jeton
`POST /m14/clients/{link}/consent` , `DELETE /m14/clients/{link}`	`client_id`
`GET/POST/DELETE /m14/chat/…`	`reader_id` , `sender_id` , `requester_id`
`GET /m14/clients/{client}/brief` , `/wardrobe` , `PATCH /m14/handoff/{id}` , `POST /m14/looks` , `/review` , `/instore`	`pro_id`

Les règles d'accès sont déjà codées dans `service.py` (`_require_consent` , `_chat_role`) : il suffit de leur passer l'identité authentifiée.

4.2 Vraie garde-robe (M1)

1. Fusionner la branche `garde-robe` (BO3, Amine).
2. Dans `backend/modules/m14_stylist_mode/config.yaml` : `real: [m7, m1]` .
3. Vérifier : la fiche affiche « Sources : M1 réel » ; `GET /m14/clients/{client}/wardrobe?pro_id=` renvoie les vraies pièces.
4. Amélioration app : la page « Aider » affiche les pièces en texte. Avec M1, chaque pièce a une photo (`image` dans `/wardrobe`) : l'afficher dans `_WardrobePicker` (`bo6_help_screen.dart`).

Rien d'autre à coder : le client M1 suit déjà le contrat `ItemsRequest` / `ItemsResponse` .

4.3 Port des vêtements (M3) et score des looks (M2)

- Fusionner `feat/m02-m03-m04-m17` ; `real: [m7, m1, m3, m2]` .
- M2 a besoin d'**au moins 2 pièces** dans le look, sinon M14 garde le score mock.
- Les pièces M7 sont converties au format `Garment` par `clients.product_as_garment` (photo = URL de la boutique, couleur + hex du catalogue). Si M2 a besoin d'embeddings pour les pièces de boutique, ils existent dans l'index M7 (même espace FashionCLIP 512) : à ajouter dans `product_as_garment` .

4.4 Profil de style (M5, BO4) — à définir ensemble

M5 n'est pas encore implémenté. M14 attend :

Appel	Réponse attendue par M14
`POST /m05/profile {user_id}`	`result: {archetype, likes[], dislikes[], colours[], style_vector?[512]}`
`POST /m05/gaps {user_id}`	`result: {gaps: ["chaussures habillées", …]}` (textes ou objets avec `label`)
`POST /m05/feedback {user_id, payload: {source:"stylist", pro_id, corrections}}`	n'importe quelle réponse 2xx

Si BO4 choisit d'autres noms de champs, il suffit d'adapter `style_profile`, `style_gaps` et `send_corrections` dans `clients.py`. Le `style_vector` sert aussi au choix du styliste (F7) et à M7.

4.5 Vendeur en magasin (M6) et cartes (M10)

- `feat/bo5-smartbuy` → `real: […, m6]`. M14 envoie la **photo de la boutique en URL http** comme `probe_image` ; le contrat de M6 l'accepte (« data URL / http »). À vérifier en vrai avec Aziz.
- `feat/bo7-m10-m15` → `real: […, m10]`. Le format `CardItem` est validé ; la carte est créée à chaque look envoyé (`card_url`, `share_url` dans la réponse).
- **M13** (liens partenaires, BO5) n'est pas encore utilisé : quand il sera fusionné, transformer les `product_url` des looks en liens partenaires (`POST /m13/links`) dans `service.send_look`.

4.6 Orchestrateur (BO8) : brancher le hand-off sur le chat

Aujourd'hui, le hand-off se déclenche seulement depuis l'app (simulateur de l'Espace pro). À faire :

1. Dans `localshop` (`m07_local_catalogue/agent.py → _next_actions`), ajouter une action `{"action": "handoff", "label": "Parler à un styliste", "tool": None, "args": {"endpoint": "/m14/handoff"}}` quand la recherche ne donne rien ou que l'occasion est importante.
2. Côté orchestrateur, quand l'utilisateur choisit cette action (ou que `POST /m14/handoff/check` renvoie `escalate: true`), appeler : `json POST /m14/handoff {"user_id": "...", "session_id": "<thread_id>", "language": "fr", "messages": [{"role": "user", "text": "..."}, {"role": "assistant", "text": "..."}], "force": true}` Les messages viennent de `ChatState.messages`. La réponse contient `status`, la styliste choisie et le résumé. Si `already_open` vaut `true`, le client a déjà une demande en cours.
3. Afficher au client « Ines (styliste, Sousse) va vous répondre » et ouvrir la conversation privée.

4.7 Événements partagés

M7 écrit déjà `local_search`, `product_view`, `shop_click` au format du schéma `Event` partagé. À ajouter dans `shared/schemas/event.py` (PR relue par M6) : `local_search`, `product_view`, `shop_click`, `handoff_request`, `handoff_closed`, `look_shared`, `profile_review`.

4.8 Stockage : JSON → Postgres

`store.py` expose `Collection.all/get/put/delete/find`. Le remplacer par des tables (`pros`, `client_links`, `handoffs`, `looks`, `reviews`, `chat_messages`) **sans toucher à** `service.py`. Même chose pour la mémoire de M7 (`memory.py`, un JSON par utilisateur).

4.9 Application

- **Espace pro** (fait) : `/bo6_boutique?tab=0|1|2`, page `/bo6_boutique/help/:pro/:client?handoff=`.
- **À faire côté client** (app client, BO7/BO8) : accepter ou refuser une invitation, voir et retirer les accès, recevoir les looks (`GET /m14/looks?client_id=`), conversation (`/m14/chat` en tant que client), bouton « Parler à un styliste ».
- **Adresse de l'API** : `--dart-define=API_URL=...`. En web, `tool/serve_app.py` sert l'app et l'API sur le même port (transmet `/m07/` et `/m14/`, GET/POST/PATCH/DELETE). Si l'app web est servie ailleurs, ajouter `CORSMiddleware` dans `backend/api/main.py` (absent aujourd'hui).

4.10 Confidentialité (déjà en place, à garder)

- Pas de données client sans consentement (403) ; le client voit et retire les accès.
- Conversation privée : la styliste et le client seulement ; fermée si l'accès est retiré ; effaçable par le client.
- Tableau de bord : agrégats seulement, aucun identifiant, groupes de moins de 10 recherches masqués.
- Ce qui manque : `DELETE` de toutes les données M14 d'un client (demandes, looks), comme `DELETE /m07/memory/{user}`.

4.11 Le LLM du BO6 : Qwen 2.5 7B en local (Ollama)

Un seul LLM pour l'agent BO6 (M7 et sa compétence M14), dans `backend/modules/m07_local_catalogue/llm.py`. Il **lit et rédige** ; il ne choisit **jamais** un produit, un prix, un score ou un styliste.

Où	Ce que fait le LLM	Ce que le code vérifie
M7 `pipeline/understand.py`	Requête que les règles ne comprennent pas du tout → type de pièce, couleur, genre, budget, ville (JSON) + reformulation en français ; puis la recherche normale	Schéma JSON avec les codes du catalogue ; budget écrit dans la requête, ville et genre dits dans la requête ; gardé seulement si la recherche trouve des pièces
M14 résumé du hand-off	Une phrase claire pour la styliste	≤ 260 caractères, aucun chiffre que le client n'a pas écrit ; les lignes de faits restent calculées
M14 escalade	Conversations longues que les règles n'escaladent pas	Le LLM doit citer les mots du client ; une citation absente de la conversation est ignorée ; il peut ajouter un signal, jamais en retirer
M14 conseils boutique (`GET /m14/dashboard/insights`)	2-3 conseils à partir des chiffres du tableau de bord	Tout chiffre d'un conseil doit exister dans les chiffres, sinon le conseil est retiré ; conseils par règles si le LLM est éteint

Installation (machine qui fait tourner l'API) :

```
# 1. installer Ollama (https://ollama.com), puis :
ollama pull qwen2.5:7b          # ~4,7 Go
ollama serve                    # si le service n'est pas déjà lancé (port 11434)
```

Réglages : `backend/modules/m07_local_catalogue/config.yaml → llm` (`enabled` , `url` , `model` , `timeout_s` , un interrupteur par usage). Pour une machine faible : `model: qwen2.5:3b` (plus rapide, un peu moins juste, voir `evaluation/RESULTS_LLM.md`).

Langue des réponses : M7 répond dans la langue demandée (`language` : `fr` par défaut, `en` , `ar`) ; ses notes et ses raisons existent en français et en anglais (l'arabe garde le français pour l'instant). M14 écrit en anglais, comme l'Espace pro de l'app. La requête du client peut toujours être en français, anglais, darija ou arabe. L'app Flutter BO6 est en anglais et envoie `language: "en"` à M7.

Sans Ollama, rien ne casse : chaque usage retombe sur les règles (même logique que réel/mock). Les tests n'appellent jamais Ollama (faux LLM).

Lien avec `shared/llm.py` (BO8) : l'orchestrateur passe par LangChain (`get_chat_model()`), le BO6 appelle directement l'API d'Ollama pour utiliser les sorties JSON contraintes par un schéma, sans dépendance de plus. Les deux peuvent utiliser le **même serveur Ollama**. Si l'équipe passe à un LLM par API, seule la méthode `LLM._chat` de `llm.py` est à adapter.

Temps de réponse mesurés sur processeur seul (i7-1165G7, 32 Go, sans GPU), avec `qwen2.5:7b` :

Usage	Temps	Avec `qwen2.5:3b`
Compréhension M7 (médiane sur 16 requêtes)	≈ 15 s	≈ 7,6 s
Résumé du hand-off	7-10 s	environ deux fois moins
Escalade (conversation longue)	9-14 s	environ deux fois moins
Conseils boutique	≈ 30 s	environ deux fois moins
Premier appel (chargement du modèle)	+ 15-20 s	+ 5-10 s

Résultat M7 (`evaluation/RESULTS_LLM.md`) : sur 16 requêtes que les règles ne comprennent pas, le type de pièce passe de **0/16 à 11/16** avec le 7B (12/16 avec le 3B), sans budget ni ville inventés. C'est pour cela que le LLM n'est appelé que lorsque les règles échouent, et que les conseils boutique sont générés à la demande (bouton « Générer » dans l'app), pas à chaque ouverture du tableau de bord.

---

5. Fusion : ordre conseillé et conflits attendus

1. **`BO6/local_catalogue` en premier** : M7 est utilisé par Smart Buy, BO7 et l'orchestrateur.
2. Puis `garde-robe` (M1), `feat/m02-m03-m04-m17` , `feat/bo5-smartbuy` , `feat/bo7-m10-m15` , `feat/bo8-orchestrator` .
3. Après chaque fusion, ajouter le module dans `real:` de M14 et relancer les tests.

Conflits attendus - `feat/bo8-orchestrator` contient une **ancienne copie** de `backend/modules/m07_local_catalogue/` et un `m14_stylist_mode/identity.py` + une route `/identity`. **Garder la version BO6** de ces deux dossiers (elle contient la sienne : même nom `localshop`, mêmes compétences, même `handoff`) : `bash git checkout BO6/local_catalogue -- backend/modules/m07_local_catalogue backend/modules/m14_stylist_mode` - `mobile/lib/app/router.dart` : garder les routes de tout le monde (BO6 ajoute `/bo6_boutique` et `/bo6_boutique/help/...`). - `shared/agent_card.py` (branche BO8) : `identity.py` de M7 l'utilise s'il existe, sinon garde son propre texte. Rien à faire.

---

6. Vérifier que tout marche

```
cd backend
pytest modules/m07_local_catalogue modules/m14_stylist_mode orchestrator   # 197 tests sur la branche BO6
python -m modules.m14_stylist_mode.simulate.pros --reset
python -m modules.m14_stylist_mode.simulate.traffic
uvicorn api.main:app --port 8000                                           # http://localhost:8000/docs

cd ../mobile
flutter build web --dart-define=API_URL=http://localhost:8090
python tool/serve_app.py --api http://127.0.0.1:8000                       # http://localhost:8090/#/bo6_boutique
```

Contrôles rapides après chaque intégration : - `GET /m14/clients/usr_042/brief?pro_id=<pro>` → `sources` montre quels modules sont réels. - Espace pro → Hand-off → « Envoyer à l'assistant » → la demande arrive chez la bonne styliste → Accepter → page Aider → envoyer un look → il apparaît dans la conversation, avec un score M2. - Tableau de bord → une boutique → la part simulée est affichée en bandeau.

---

7. Réglages (`backend/modules/m14_stylist_mode/config.yaml`)

Clé	Rôle
`real`	modules appelés pour de vrai (`[m7]` aujourd'hui)
`api_url`, `timeout_s`	adresse de l'API pour les appels HTTP vers les autres modules
`min_group`	taille minimale d'un groupe affiché dans le tableau de bord (confidentialité)
`dashboard.*`	période, sources, nombre de thèmes, seuils de tendance, couverture du stock
`handoff.*`	seuil et poids de l'escalade, poids du choix du styliste, charge maximale

8. Contacts

Module	Branche	Personne
M7, M14 (BO6)	`BO6/local_catalogue`	kenzabokri
M1 Garde-robe (BO3)	`garde-robe`	aminebali

Module	Branche	Personne
M2, M3, M4, M17 (BO1/BO2)	`feat/m02-m03-m04-m17`	Raecm
M6, M13, M18 (BO5)	`feat/bo5-smartbuy`	Mohamed Aziz Chouaia
M10, M15 (BO7)	`feat/bo7-m10-m15`	Omar Ferchichi
M8, orchestrateur (BO8)	`feat/bo8-orchestrator`	Nadhir Maamar
M5 (BO4)	— (pas encore de branche)	à confirmer

Documentation détaillée : `backend/modules/m07_local_catalogue/README.md`, `backend/modules/m14_stylist_mode/README.md`, `rapport/rapport_M7_catalogue_local_FR.pdf`, `rapport/rapport_M14_stylist_boutique_FR.pdf`.